

# ROD-256: C++ Tabanlı Deneyisel Karma Algoritması ile Bulut Tabanlı Kimlik Doğrulama Sistemi

Rodina Amr Mukhtar

Bilgisayar Mühendisliği Bölümü, Kastamonu Üniversitesi

Kastamonu, Türkiye

0009-0002-0696-0929

**Öz**—Bu çalışmada, nesne yönelimli C++ programlama dili kullanılarak sıfırdan tasarlanan ROD-256 adlı özgün bir karma (hash) algoritması ve bu algoritmanın temel bileşen olarak kullanıldığı tam yığın bulut tabanlı güvenli bir kimlik doğrulama sistemi sunulmaktadır. ROD-256; sekiz adet 32-bit dahili durum değeri, 64 karıştırma turu, XOR işlemleri, bit döndürme (rotateLeft/rotateRight) ve modüler toplama kullanarak 256-bit özet üretmektedir. Her giriş baytı için beş aşamalı bir karıştırma süreci uygulanmakta ve çok aşamalı difüzyon sayesinde çıktı hassasiyeti hedeflenmektedir. Sistem; RSA imzalı JWT kimlik doğrulama mekanizması, bcrypt parola hashleme, Supabase PostgreSQL bulut veritabanı entegrasyonu, karma zincirli denetim defteri ve SHA/RIPEMD algoritmalarıyla yapısal karşılaştırma bileşenlerini bünyesinde barındırmaktadır. React ve Vite ile geliştirilen ön yüz Vercel'de, Node.js ve Express.js ile inşa edilen arka uç ise Render bulut platformunda konuşlandırılmıştır.

**Anahtar Kelimeler**—karma algoritması; deneyisel kimlik doğrulama; C++ sistem programlaması; RSA-JWT; karma zincirli denetim defteri; bulut güvenliği; ROD-256; bit döndürme; XOR difüzyonu

## I. GİRİŞ

Dijital sistemlerin hızla yaygınlaşması ve çevrimiçi hizmetlerin gündelik yaşamın ayrılmaz bir parçası haline gelmesiyle birlikte kullanıcı kimlik doğrulama sistemlerinin güvenliği kritik bir önem kazanmıştır. Parola tabanlı kimlik doğrulama hâlâ en yaygın yöntem olmakla birlikte, karma algoritmalarının tasarımındaki güvenlik açıkları veri ihlallerine zemin hazırlayabilmektedir [1]. Bu bağlamda, kimlik doğrulama altyapılarının hem kriptografik hem de mühendislik boyutlarını ele alan bütünlük bir yaklaşıma ihtiyaç duyulmaktadır.

Bu çalışma, Kastamonu Üniversitesi Bilgisayar Mühendisliği Bölümü bünyesindeki bir sistem programlaması projesi kapsamında geliştirilmiş ve ROD-256 adıyla özgün bir karma algoritması tasarlanmıştır. Algoritma, nesne yönelimli C++ programlama diliyle sıfırdan inşa edilmiş olup derlenen çalıştırılabilir dosya aracılığıyla Node.js arka ucu ile entegre edilmiştir. Sistem; Vercel üzerindeki React ön yüzünden, Render platformundaki Express.js arka ucuna ve Supabase PostgreSQL bulut veritabanına uzanan kapsamlı bir bulut mimarisini kapsamaktadır.

Bu çalışmanın temel katkıları şu şekilde özetlenebilir:

- Nesne yönelimli C++ ile tasarlanmış, beş aşamalı karıştırma turu kullanan orijinal bir karma algoritması (ROD-256) sunulmaktadır.
- Sistem programlaması ile modern bulut mimarisini bir araya getiren tam yığın bir kimlik doğrulama sistemi gerçekleştirilmiştir.
- ROD-256, SHA ailesi ve RIPEMD-160 algoritmaları ile karşılaştırmalı olarak değerlendirilmiştir.
- Her kimlik doğrulama olayı için kriptografik bütünlük doğrulamasını destekleyen karma zincirli (hash-chained) bir denetim defteri tasarlanmıştır.
- ROD-256'nın canlı animasyonlu görselleştirmesini içeren etkileşimli ve halka açık bir web uygulaması yayına alınmıştır.

## II. İLGİLİ ÇALIŞMALAR

Karma algoritmalarının kriptografik tasarımı ve kimlik doğrulama sistemlerindeki güvenlik mekanizmaları literatürde yoğun biçimde araştırılmaktadır. MD5 ve SHA-1 gibi erken dönem algoritmalar çeşitli saldırılara karşı savunmasız olduğu kanıtlanmıştır;

SHA-2 ve SHA-3 bu açıkları gidermek amacıyla NIST tarafından standart olarak kabul edilmiştir [2].

Parola depolama alanında bcrypt [3], scrypt [4] ve Argon2 [5] gibi anahtar üretme fonksiyonları, bellek yoğun karma yapıları aracılığıyla kaba kuvvet saldırılarına karşı güvence sağlamaktadır. Bhatt ve Patil [6], çok faktörlü doğrulama katmanlarının kimlik doğrulama güvenliğine olan katkısını somut ölçümlerle ortaya koymuşlardır.

JWT tabanlı kimlik doğrulama mekanizmaları durumsuz oturum yönetimi için yaygın biçimde tercih edilmektedir [7]. RS256 imzalama yöntemi, HS256'ya kıyasla daha güçlü bir güvenlik garantisi sunmaktadır [8]. Blok zinciri tarzı denetim defterlerinin uygulama günlüğü doğrulamasında kullanımı giderek artan bir ilgi görmektedir; her kaydın önceki girişin özetini barındırması sayesinde geçmişe dönük değişiklikler tespit edilebilmektedir [9]. ROD-256 sistemi bu birikimi sürdürerek deneyisel bir algoritma tasarımını tam yığın bir platform üzerinde gerçekleştirmektedir [10].

## III. SİSTEM MİMARİSİ

ROD-256 Deneyisel Karma Algoritması Kimlik Doğrulama Sistemi, birbirleriyle iletişim kuran dört temel katmandan oluşmaktadır: ön yüz istemcisi, arka uç API, C++ karma motoru ve bulut veri depolama katmanı.

### A. Bulut Mimarisi

Ön yüz, Vercel CDN altyapısı üzerinde statik bir React uygulaması olarak sunulmaktadır. Kimlik doğrulama istekleri Render platformundaki Node.js/Express.js arka ucuna yönlendirilmektedir. Arka uç, karma işlemleri için derlenmiş C++ CLI motorunu çağırmakta ve oturum verilerini Supabase PostgreSQL'de saklamaktadır.

### B. Kimlik Doğrulama Akışı

Sistem akışı yedi adımdan oluşmaktadır:

- Kullanıcı React ön yüzüne erişir.
- Kayıt veya giriş isteği Render arka ucuna iletilir.
- Parolalar, veritabanına kaydedilmeden önce bcrypt ile hashlenir.
- Başarılı girişte RSA özel anahtarıyla imzalanmış bir JWT üretilir.
- Kimlik doğrulama olayı, ROD-256 tarafından hashlenerek karma zincirli denetim defteri bloğuna yazılır.
- Veriler Supabase PostgreSQL'e aktarılır.

7. Kullanıcı defteri doğrulayabilir ve karma algoritmalarını test edebilir.

## IV. ROD-256 KARMA ALGORİTMASI

### A. Genel Tasarım Felsefesi

ROD-256, SHA ailesi karma algoritmalarındaki temel kriptografik ilkeleri — özellikle ARX (Toplama-Döndürme-XOR) yapısını — özgün bir çerçevede yeniden yorumlayan eğitim odaklı bir algoritmadır. SHA-256 ile paylaştığı 256-bit çıktı boyutu ve Nothing-up-my-sleeve (NUMS) başlangıç sabit seçim ilkesinin yanı sıra ROD-256, her karakter için bağımsız çok-turlu karıştırma uygulayan seri bir işleme modelini benimser. Bu tercih, implementasyon şeffaflığını ve pedagojik izlenebilirliği ön plana çıkarmaktadır.

### B. Veri Yapısı ve Durum Temsili

Algoritmanın temel veri yapısı, Rod256Hasher sınıfı içinde tanımlanan sekiz elemanlı bir `uint32_t` dizisidir. Her eleman 32 bit genişliğindedir; sekiz elemanın bütünü  $8 \times 32 = 256$  bit, yani algoritmanın özet boyutunu oluşturmaktadır. Durum dizisi, pi sayısının onaltılık açılımından alınan NUMS sabitleriyle başlatılmaktadır:

```
// rod256.cpp - resetState()
state = {
    0x243F6A88, 0x85A308D3,
    0x13198A2E, 0x03707344,
    0xA4093822, 0x299F31D0,
    0x082EFA98, 0xEC4E6C89
};
```

### C. Tur Sabiti Üretimi

Her tur için benzersiz bir sabit üretilmektedir. Bu sabit, Golden Ratio'nun onaltılık karşılığı olan `0x9E3779B9` değerinin tur indeksine bağlı bir çarpanla XOR'lanmasıyla hesaplanmaktadır:

```
// rod256.cpp - roundConstant(int round)
return 0x9E3779B9 ^ (round * 0x7F4A7C15);
```

Bu yaklaşım, her tura özgü farklı sabit değerler üretmek ve tur çıktıların ayrıştırılmasına katkı sağlamayı amaçlamaktadır. `0x9E3779B9` sabiti, TEA ve Blowfish gibi köklü blok şifrelemelerinde de kullanılan tanınmış bir kriptografik sabit olup irrasyonel sayıların öngörülmesi güç doğasından yararlanmaktadır.

### D. Beş Aşamalı Karıştırma Turu — `mixCharacter()`

ROD-256'nın kalbini oluşturan `mixCharacter()` fonksiyonu, her giriş baytı için 64 kez çalışmakta ve her turda beş ayrı işlem uygulamaktadır. Aşağıda tam C++ implementasyonu yer almaktadır:

```
// rod256.cpp - mixCharacter(uint32_t value)
for (int round = 0; round < ROUNDS; round++) {
    int index = round % STATE_SIZE;

    // Adım 1: İlk XOR karistirmasi
    uint32_t mix = state[index]
        ^ value
        ^ roundConstant(round);

    // Adım 2: Sol dondurmeli yayilim
    mix = rotateLeft(mix, 7);

    // Adım 3: Komsuz durum toplama
```

```
mix = mix + rotateRight(
    state[(index+1) % STATE_SIZE],
    11);

    // Adım 4: Uzak durum XOR difuzyonu
    mix = mix ^ rotateLeft(
        state[(index+3) % STATE_SIZE],
        17);

    // Adım 5: Durum guncelleme (cift katmanli)
    state[index] = state[index] ^ mix;
    state[index] = state[index]
        + rotateLeft(value + round,
            (round % 31) + 1);
}
```

Her adımın rolü aşağıda açıklanmaktadır:

- Adım 1 — İlk XOR Karıştırması: Mevcut durum değeri (`state[index]`), giriş baytı (`value`) ve tur sabiti XOR işlemiyle birleştirilir. Bu üçlü XOR, giriş bilgisini durum içine enjekte eder.
- Adım 2 — Sol Döndürme (`rotL 7`): Karıştırma değeri 7 bit sola döndürülür. Bit döndürme, bilgi kaybı olmadan bit düzeyinde konum değiştirme sağlar ve doğrusal olmayan yayılımı destekler.
- Adım 3 — Komşu Durum Toplama: Bir sonraki durum hücresi (`index+1`) sağa 11 bit döndürülerek modüler olarak toplanır. Bu, bitişik durum hücrelerini birbirine bağlayan bir çapraz karıştırma etkisi yaratır.
- Adım 4 — Uzak Durum XOR Difüzyonu: Üç adım uzaktaki durum hücresi (`index+3`) sola 17 bit döndürülerek XORlanır. Bu, tek bir tur içinde geniş mesafeli bağımlılık kurarak çıg etkisine katkıda bulunur.
- Adım 5 — Çift Katmanlı Güncelleme: Durum hücresi önce `mix` ile XORlanır, ardından tur-bağımlı değişken bir döndürme miktarıyla (`round % 31 + 1`) döndürülen giriş toplamıyla artırılır. Değişken döndürme miktarı, sabit döndürme miktarlı tasarımlardaki periyodikliği kırmaktadır.

### E. Özet Üretimi — `hash()`

Tüm giriş baytları işlendikten sonra sekiz durum değeri onaltılık biçimde dizgeye dönüştürülerek birleştirilmektedir. Bu süreç aşağıdaki gibi gerçekleşmektedir:

```
// rod256.cpp - hash()
std::string data = message + salt;
for (char ch : data) {
    uint32_t val =
        static_cast<uint32_t>(
            static_cast<unsigned char>(ch));
    mixCharacter(val);
}

// Digest olusturma
for (uint32_t part : state) {
    result << hex << setw(8)
        << setfill('0') << part;
}
```

Mesaj ve tuz önce birleştirilerek tek bir veri akışı oluşturulmaktadır. Her karakter bayt değerine (0–255) dönüştürülmekte ve `unsigned char` kullanımıyla işaret uzantısı sorunu önlenmektedir. Sekiz durum değerinin her biri 8 onaltılık

karaktere (32 bit / 4 bit = 8) karşılık gelmekte ve bu adım toplamda 64 karakterlik 256-bit özetı üretmektedir.

#### F. Tuz Mekanizması ve Güvenlik Etkisi

ROD-256, tasarımı gereği tuz (salt) parametresini zorunlu kılmaktadır. Mesaj ve tuz birleştirilerek tek bir veri akışı olarak işlenmektedir. Bu yaklaşımın iki önemli etkisi vardır: (1) Aynı mesaj, farklı tuzlarla farklı özetler üretir; bu sayede önceden hesaplanmış sözlük saldırıları (rainbow table) etkisizleştirilir. (2) Tuz değerinin her kullanıcı için benzersiz seçilmesi, aynı parolayı kullanan farklı kullanıcıların veritabanında farklı özetlere sahip olmasını sağlar.

#### G. Örnek Çıktı

Aşağıdaki girdi çifti için ROD-256 üretilen özet değeri şu şekildedir:

```
Mesaj : password123
Tuz   : ABC987
ROD-256:
fbd11d248c0f19adf7efab23ed0b92b8
db8bff25c00b34afbfc1cf3843a8ea9d
```

64 onaltılık karakter x 4 bit = 256 bit. Sekiz 32-bit durum değeri sırasıyla şu onaltılık bloklara karşılık gelmektedir: fbd11d24, 8c0f19ad, f7efab23, ed0b92b8, db8bff25, c00b34af, bfc1cf38, 43a8ea9d.

## V. GÜVENLİK BİLEŞENLERİ

#### A. RSA İmzalı JWT Kimlik Doğrulama

Sistem, RS256 (RSA + SHA-256) algoritmasını kullanarak belirteç imzalama işlemini gerçekleştirmektedir. RSA anahtar çifti scripts/generateKeys.js betiği aracılığıyla üretilmektedir; özel anahtar belirteci imzalamak, açık anahtar ise doğrulamak için kullanılmaktadır. Bu yaklaşım, HS256'ya kıyasla çok daha güçlü bir güvenlik modeli sunmakta; kimlik doğrulama ve doğrulama için iki farklı anahtar kullanılmaktadır.

#### B. bcrypt Parola Hashleme ve ROD-256'nın Rolü

Kullanıcı parolaları, veritabanına kaydedilmeden önce bcrypt algoritmasıyla hashlenmektedir. bcrypt'in uyarlanabilir iş faktörü mekanizması, donanım hızlanmasına karşı sistematik bir direnç sağlamaktadır. Bu sayede ham parolalar hiçbir zaman depolanmamakta; olası bir veritabanı sızıntısında bile hassas veriler korunmuş olmaktadır. Önemli bir not olarak belirtmek gerekir ki ROD-256, parola depolamada bcrypt'in yerini almamaktadır; ROD-256'nın sistemdeki temel işlevi denetim defteri bloklarını hashlemektir. bcrypt, parola güvenliği için endüstri standardı olmayı sürdürmektedir.

#### C. Karma Zincirli Denetim Defteri

Her kayıt ve giriş olayı audit ledger tablosuna bir blok olarak eklenmektedir. Her blok; blok indeksi, kullanıcı adı, olay türü, durum bilgisi, IP adresi, zaman damgası, önceki bloğun özetini ve ROD-256 ile üretilmiş mevcut bloğun özetini içermektedir. Bu yapı bir blok zinciri (blockchain) değil, her kaydın bir öncekinin özetini taşıdığı karma zincirli (hash-chained) bir denetim mekanizmasıdır; merkezi bir veritabanı üzerinde çalışmakta olup tam anlamıyla dağıtık değildir. Herhangi bir bloğun içeriği değiştirildiğinde, sonraki tüm blokların doğrulama süreci başarısız olmakta; böylece kurcalamalar tespit edilebilmektedir.

#### E. Tehdit Modeli

Sistemin güvenlik kapsamını netleştirmek amacıyla kısa bir tehdit modeli sunulması gerekmektedir. Sistem şunları

korumaktadır: kullanıcı parolaları bcrypt aracılığıyla kaba kuvvet saldırılarına karşı korunmakta; oturum belirteçleri RS256 ile imzalanarak taklit saldırılarına karşı güvence altına alınmakta; denetim defteri kayıtları karma zincirleme ile değişiklik tespitine olanak tanımaktadır. Sistem şunları garanti etmemektedir: ROD-256'nın çarpışma direnci veya ön görüntü direnci kriptografik olarak kanıtlanmamıştır; merkezi veritabanına doğrudan erişim halinde kayıt geçmişi değiştirilebilir; ROD-256, üretim ortamında bcrypt veya SHA-256'nın yerini alamaz. Bu sınırların açıkça tanımlanması, projenin eğitim odaklı niteliğiyle örtüşmekte ve güvenlik beklentilerini gerçekçi bir çerçevede sunmaktadır.

Supabase PostgreSQL; TLS şifreli bağlantılar, satır düzeyinde güvenlik politikaları ve otomatik yedekleme mekanizmaları sunmaktadır. Sistem, DATABASE\_URL ortam değişkeninin tanımlı olmaması durumunda yerel JSON depolamaya otomatik olarak geçiş yapabilmektedir.

## VI. FRONTEND UYGULAMASI VE KULLANICI DENEYİMİ

React/Vite ile geliştirilen ön yüz dört ana sayfadan oluşmaktadır: Ana Sayfa, Konsol, Algoritma ve Kontrol Paneli. Glassmorphism arayüz dili ve kahve teması esas alınmış; açık/koyu tema desteği Bootstrap 5 aracılığıyla hayata geçirilmiştir.

#### A. Canlı Algoritma Görselleştirmesi

Algoritma sayfası, ROD-256'nın beş karıştırma adımını gerçek zamanlı olarak canlandırmaktadır. Kullanıcı mesaj ve tuz girdikçe animasyonlu kahve fincanları sekiz dahili durum değerinin değişimini görsel olarak yansıtmaktadır. Arayüz, her adımı — bayta dönüştürme, durum karıştırma, döndürme ve özet üretme — ayrı görsel panellerle sunmaktadır.

#### B. Karma Karşılaştırma Panosu

Canlı Kriptografik Karşılaştırma özelliği, aynı mesaj ve tuz için SHA-256, RIPEMD-160 ve SHA3-256 çıktılarını eş zamanlı olarak göstermektedir. Kullanıcılar algoritmalara özgü özellikleri ve çıktı farklılıklarını doğrudan gözlemleyebilmektedir.

#### C. Kontrol Paneli

Oturum açan kullanıcılara yönelik Kontrol Paneli, JWT belirteci ayrıntılarını, blok bütünlüğü doğrulama sonuçlarını ve denetim defteri blok zinciri durumunu gerçek zamanlı olarak görüntülemektedir.

## VII. KARMA ALGORİTMALARININ KARŞILAŞTIRMALI ANALİZİ

Sistem, ROD-256 ile SHA ve RIPEMD ailelerine ait algoritmalar arasında yapısal karşılaştırma gerçekleştirmektedir. Tablo I, desteklenen algoritmaların temel yapısal özelliklerini özetlemektedir; kantitatif performans ve çıg etkisi ölçümleri ilerideki çalışmalar için planlanmaktadır.

| Algoritma  | Çıktı   | Yapı / Özellik                    |
|------------|---------|-----------------------------------|
| ROD-256    | 256 bit | ARX + Döngüsel durum, tuz zorunlu |
| SHA-256    | 256 bit | Merkle-Damgård, 64 tur            |
| SHA3-256   | 256 bit | Sünger (Keccak), 1600-bit durum   |
| SHA-512    | 512 bit | Merkle-Damgård, 80 tur            |
| RIPEMD-160 | 160 bit | Çift paralel Merkle-Damgård       |

| Algoritma | Çıktı   | Yapı / Özellik             |
|-----------|---------|----------------------------|
| SHA-1     | 160 bit | Merkle-Damgård (kıvrılmış) |

Tablo 1. Desteklenen Karma Algoritmalarının Karşılaştırması

ROD-256, tuz parametresini zorunlu kılan yapısıyla ve çok aşamalı çapraz durum difüzyonuyla genel amaçlı SHA algoritmalarından ayrılmaktadır. Bununla birlikte biçimsel kriptografik doğrulamadan henüz geçirilmemiş olması nedeniyle yalnızca eğitim amaçlı kullanım için önerilmektedir.

## VIII. API UÇLARI VE ARKA UÇ YAPISI

RESTful API, Express.js üzerinde üç yönlendirici grubundan oluşmaktadır: kimlik doğrulama (authRoutes), denetim defteri (ledgerRoutes) ve kriptografi (cryptoRoutes). Sistemin başlıca uçları şunlardır: POST /api/auth/register, POST /api/auth/login, POST /api/auth/verify-token, POST /api/rod256/hash, POST /api/crypto/compare, GET /api/ledger ve GET /api/ledger/verify.

Sistem, DATABASE\_URL değişkeninin varlığını dinamik olarak algılayarak PostgreSQL bulut depolama ve yerel JSON depolama arasında otomatik geçiş yapmaktadır. Bu tasarımı, hem tam bulut konuşlandırma hem de yerel geliştirme ortamında sorunsuz çalışmayı mümkün kılmaktadır.

## IX. BULUT KONUŞLANDIRMASI

Proje üç farklı bulut platformuna dağıtılmış çok katmanlı bir konuşlandırma yapısına sahiptir. Vercel ön yüzü, Render arka ucu barındırmakta ve Supabase PostgreSQL veritabanı hizmetini sağlamaktadır. Arka uç derleme komutu (cd backend && npm install && npm run build), g++ derleyicisi ile C++ ROD-256 kaynak kodunu platform bağımsız biçimde derlemektedir; Windows'ta .exe, Render'ın Linux ortamında ELF ikili üretilmektedir.

## X. TARTIŞMA

### A. Güçlü Yönler

ROD-256 sistemi birçok teknik açıdan öne çıkmaktadır. Beş aşamalı karıştırma turunun her adımı, farklı bir difüzyon mekanizması (XOR, modüler toplama, değişken döndürme) uygulanarak çıktının hem giriş hem de tur indeksine bağımlı olmasını sağlamayı amaçlamaktadır. Sistem programlamasının modern bulut geliştirme ile bütünleştirilmesi özgün bir yaklaşım sunarken karma zincirli denetim defteri, güvenlik kayıtlarına izlenebilirlik kazandırmaktadır.

### B. Sınırlılıklar

ROD-256'nın en önemli sınırlılığı biçimsel kriptografik analize henüz tabi tutulmamış olmasıdır. Çarpışma direnci, ön görüntü direnci ve diferansiyel kriptanalize yönelik dayanıklılık değerlendirmeleri gerçekleştirilmemiştir. Ayrıca seri işleme modeli, paralel veri yolu üzerinde çalışan SHA-256 donanım hızlandırmasıyla rekabet edecek performans hedeflemiş değildir.

## XI. ARAŞTIRMA ÖNERİLERİ

Bu araştırma bulguları doğrultusunda aşağıdaki öneriler öne çıkmaktadır:

1. ROD-256 algoritması, çarpışma direnci, ön görüntü direnci ve diferansiyel saldırı direncini kapsayan biçimsel kriptografik analize tabi tutulmalıdır.
2. Çığ etkisi istatistiksel yöntemlerle ölçülmeli; tek bit giriş değişikliğinin kaç çıktı bitini etkilediği hesaplanmalıdır.
3. Sistem, çok faktörlü kimlik doğrulama (MFA), e-posta doğrulama ve hesap kilitleme gibi güvenlik katmanlarıyla genişletilmelidir.
4. Yenileme belirteci mekanizması eklenmeli ve JWT yaşam döngüsü yönetimi güçlendirilmelidir.
5. Karma zincirli denetim defteri, merkezi olmayan bir mimariye entegre edilerek tek hata noktası riski azaltılmalıdır.
6. ROD-256'nın yeniden kullanımını kolaylaştırmak amacıyla NPM üzerinde bağımsız bir JavaScript/WebAssembly kitaplığı geliştirilmelidir.
7. Dağıtık yük koşullarında sistemin dayanıklılığını doğrulamak için kapsamlı performans ve stres testleri gerçekleştirilmelidir.

## XII. SONUÇ

Bu çalışmada, C++ ile geliştirilmiş deneyisel bir karma algoritmasını (ROD-256) modern bulut tabanlı kimlik doğrulama mimarisine entegre eden bir prototip sistem sunulmuştur. ROD-256'nın beş aşamalı karıştırma turu — XOR tabanlı ilk birleştirme, sol döndürme, komşu durum toplama, uzak durum XOR difüzyonu ve çift katmanlı güncelleme — bir arada uygulanarak çok boyutlu bir difüzyon yapısı hedeflenmektedir.

Halka açık biçimde yayına alınan sistem (<https://rod-256-auth-system.vercel.app>), canlı karma görselleştirme, yapısal karşılaştırma panosu ve doğrulanabilir denetim defteri aracılığıyla araştırmacılar ve öğrenciler için etkileşimli bir öğrenme ortamı oluşturmaktadır. Bu çalışma, öğretim ortamlarına yönelik özel kriptografik bileşenler tasarlamak isteyen geliştiriciler için incelenebilir ve geliştirilebilir bir eğitim modeli sunmaktadır.

## TEŞEKKÜRLER

Bu çalışma, Kastamonu Üniversitesi Bilgisayar Mühendisliği Bölümü bünyesindeki Sistem Programlaması dersi kapsamında geliştirilmiştir. Araştırmacı, akademik destek ve rehberlik için bölüm öğretim üyelerine içtenlikle teşekkür etmektedir. Kullanılan açık kaynak araçlar için Node.js, React ve Supabase topluluklarına da ayrıca şükran borçluyuz.

## KAYNAKLAR

- [1] P. C. van Oorschot ve M. J. Wiener, "Parallel collision search with cryptanalytic applications," *Journal of Cryptology*, cilt 12, sayı 1, ss. 1-28, 1999. <https://doi.org/10.1007/PL00003816>
- [2] National Institute of Standards and Technology (NIST), "Secure Hash Standard (SHS)," *Federal Information Processing Standards Publication 180-4*, 2015. <https://doi.org/10.6028/NIST.FIPS.180-4>
- [3] N. Provos ve D. Mazieres, "A future-adaptable password scheme," *USENIX Annual Technical Conference*, ss. 81-91, 1999.
- [4] C. Percival ve S. Josefsson, "The scrypt password-based key derivation function," *RFC 7914*, IETF, 2016. <https://doi.org/10.17487/RFC7914>
- [5] A. Biryukov, D. Dinu ve D. Khovratovich, "Argon2: Memory-hard function for password hashing," *IEEE Transactions on Information Forensics and Security*, cilt 16, ss. 3747-3758, 2021. <https://doi.org/10.1109/TIFS.2021.3111026>

- [6] S. Bhatt ve R. K. Patil, "Multi-factor authentication in securing web applications against cyber threats," *Int. Journal of Computer Science and Information Security*, cilt 17, sayı 4, ss. 45-52, 2019.
- [7] M. Jones, J. Bradley ve N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015. <https://doi.org/10.17487/RFC7519>
- [8] M. Jones ve J. Hildebrand, "JSON Web Encryption (JWE)," RFC 7516, IETF, 2015. <https://doi.org/10.17487/RFC7516>
- [9] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Cevrimici]. <https://bitcoin.org/bitcoin.pdf>
- [10] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8. baskı. Hoboken, NJ: Pearson, 2022.
- [11] B. Schneier, *Applied Cryptography: Protocols, Algorithms and Source Code in C*, 2. baskı. New York: Wiley, 1996.
- [12] H. Krawczyk ve P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, IETF, 2010. <https://doi.org/10.17487/RFC5869>
- [13] Supabase Inc., "Supabase documentation," 2024. [Cevrimici]. <https://supabase.com/docs>
- [14] Vercel Inc., "Vercel platform documentation," 2024. [Cevrimici]. <https://vercel.com/docs>
- [15] R. A. Mukhtar, "ROD-256 Auth System - GitHub Repository," 2025. [Cevrimici]. <https://github.com/RodinaAmrMukhtar/rod-256-auth-system>